

# Генератор случайных чисел МК61s mini

Версия документа: 18.07.2026

## Назначение

У оригинального алгоритма К СЧ есть две проблемы для современного применения: после холодного старта калькулятор начинает с одного и того же внутреннего состояния, а измеренная последовательность быстро переходит в короткий цикл.

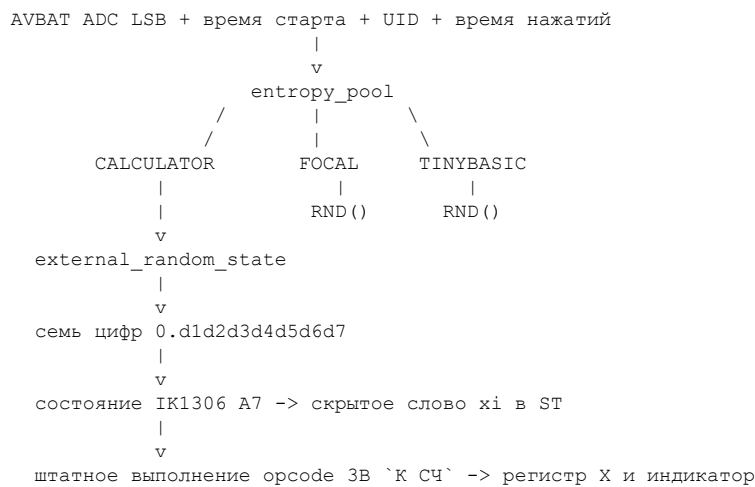
В МК61s mini предусмотрены два режима:

| Настройка  | Работа команды К СЧ                                                                                                             |
|------------|---------------------------------------------------------------------------------------------------------------------------------|
| К СЧ МК61  | Полностью штатный алгоритм и начальное состояние эмулируемого МК-61                                                             |
| К СЧ МК61s | Аппаратно инициализированный поток подставляет новое скрытое слово $x_i$ , после чего его обрабатывает штатная ROM-команда К СЧ |

FOCAL и TinyBASIC всегда используют собственные аппаратно инициализированные потоки. Настройка К СЧ на них не влияет.

Эта реализация предназначена для вычислений, моделирования и игр. Она не является криптографическим генератором и не должна применяться для ключей, паролей и других задач, где требуется криптографическая стойкость.

## Общая схема



Здесь важно различать несколько уровней:

- 1 Пул смешивает физические измерения и временные события.
- 2 Из пула создаются независимые 64-битные состояния потоков.
- 3 Поток калькулятора инициализирует external\_random\_state ядра.
- 4 Перед каждым К СЧ из него получается семизначное слово  $x_i$ .
- 5 Итог в регистре X вычисляет штатный алгоритм МК-61. Он не равен напрямую выходу 64-битного генератора.

# Сбор начального материала

## Запуск во время заставки

`entropy_pool::begin()` вызывается до показа заставки. В каждом цикле ожидания заставки `entropy_pool::poll_startup()` выполняет одно измерение. Поэтому время, которое раньше тратилось только на показ логотипа, одновременно используется для сбора начального материала.

После заставки вызывается `entropy_pool::finish_startup()`. Если пользователь пропустил заставку и данных еще мало, функция продолжает измерения до одного из двух условий:

- получено 64 бита после обработки фон Неймана;
- выполнено 1024 сырых измерения.

Затем восстанавливается обычная разрядность АЦП и создаются рабочие потоки. Число 64 здесь означает целевое количество извлеченных битов, а не доказанную оценку физической энтропии устройства.

## Источник AVBAT

Для измерений используется 12-битный АЦП на входе AVBAT. Первое чтение после переключения внутреннего мультимплексора отбрасывается. Для каждого следующего чтения в пул целиком подмешиваются:

- сырое значение АЦП;
- значение `micros()`;
- номер измерения.

Для отдельного извлечения бита берется младший бит результата АЦП. Он может иметь смещение, поэтому пары младших битов обрабатываются методом фон Неймана:

```
00 -> отбросить
11 -> отбросить
01 -> 0
10 -> 1
```

Даже отброшенные пары не пропадают полностью: сырое значение и время чтения уже были подмешаны в состояние пула.

## Дополнительные значения

При старте также подмешиваются `micros()`, `millis()` и, если платформа предоставляет `UID_BASE`, 96-битный идентификатор микроконтроллера. `UID` является постоянным и сам по себе не создает случайность, но разделяет состояния разных экземпляров устройства.

Смешивание использует 64-битную avalanche-функцию с финализатором `SplitMix64`. Она хорошо распространяет изменение входных битов по всему состоянию, но не превращает систему в криптографический генератор.

## Независимые потоки

После завершения стартового сбора из общего пула создаются три состояния с разными доменными константами:

| Домен      | Потребитель                                |
|------------|--------------------------------------------|
| CALCULATOR | Расширенный режим К СЧ МК61s               |
| FOCAL      | Функция <code>RND()</code> языка FOCAL     |
| TINYBASIC  | Функция <code>RND()</code> языка TinyBASIC |

Чтение из одного потока не сдвигает два остальных. Поэтому запуск программы FOCAL не меняет последовательность К СЧ, а вызовы К СЧ не расходуют числа TinyBASIC.

Каждый поток развивается как SplitMix64: к состоянию прибавляется константа 0x9E3779B97F4A7C15, затем применяется avalanche-функция. Интерфейс `next_u32()` сворачивает полученное 64-битное значение в 32 бита.

## Учет времени нажатий

Каждое физическое нажатие клавиши добавляет в пул:

- код клавиши;
- отметку времени события в микросекундах;
- дополнительное текущее значение `micros()`.

Событие обновляет состояния всех трех доменов. Если включен К СЧ МК61s, для ядра калькулятора дополнительно формируется новый 64-битный материал и подмешивается в `external_random_state`.

Таким образом, начальное состояние появляется еще на заставке, а человеческие задержки между дальнейшими нажатиями продолжают изменять потоки. Нажатие обычной клавиши не показывает и не записывает случайное число в видимые регистры.

## Выбор и хранение режима

Пункт находится в меню Настройки и показывает активное значение:

- К СЧ МК61;
- К СЧ МК61s.

Выбор хранится одним битом `random_mode_mk61s` в энергонезависимых настройках. При загрузке прошивки режим восстанавливается, а после запуска ядра вызывается `entropy_pool::configure_calculator()`.

Переключение в меню применяется сразу:

- для МК61s ядро получает новый 64-битный материал из домена CALCULATOR;
- для МК61 внешний hook отключается.

Если переключить МК61s обратно в МК61 без сброса, уже измененное скрытое слово `xi` остается внутри эмулируемой микросхемы, а дальше работает штатный алгоритм от этого состояния. После сброса с сохраненной настройкой МК61 полностью восстанавливаются штатное начальное состояние и последовательность.

## Использование универсального ROM-hook API

К СЧ МК61s реализован как встроенный клиент общего API перехвата ROM-команд. В основном цикле эмулятора нет отдельной проверки генератора.

В режиме МК61s ядро регистрирует в зарезервированном слоте callback `IK1306:A7`. Перед декодированием он записывает новое `xi` в транзитный ST, а следующий микрошаг выполняет штатный алгоритм К СЧ без подмены ROM-команды.

При выборе К СЧ МК61 callback удаляется. Восемь публичных регистраций API остаются доступны независимо от режима генератора.

Полный контракт API, регистрация нескольких команд, порядок callback и правила доступа к R и ST описаны отдельно в `doc/src/MK61s-mini-ROM-Hooks.md` и `doc/MK61s-mini-ROM-Hooks.pdf`.

## Как число попадает в калькулятор

### 1. Инициализация состояния ядра

При включении режима МК61s пул выдает два 32-битных слова домена CALCULATOR. Они объединяются в 64-битный `seed_material`:

```
seed_material = (next_u32(CALCULATOR) << 32)
                | next_u32(CALCULATOR)
```

Ядро смешивает его с отдельной константой и сохраняет результат в `external_random_state`. Это состояние скрыто от пользователя и не является регистром МК-61.

## 2. Обнаружение команды К СЧ

При включении К СЧ МК61s ядро регистрирует внутренний callback для IK1306:A7. На каждом цикле общий диспетчер вызывается перед чтением очередного командного слова. Callback ГСЧ срабатывает только при двух условиях:

```
external_random_enabled == true  
chip == IK1306 и address == 0xA7
```

Адрес A7 соответствует моменту, когда внутренний ROM-переход, достигнутый при выполнении пользовательского opcode 0x3B (К СЧ), готовится прочитать прежнее скрытое слово `xi`. До этого момента прошивка не трогает внутреннее состояние генератора калькулятора. Обычные команды поэтому не получают случайное значение и работают как раньше.

## 3. Получение семи десятичных цифр

При попадании в A7 состояние сдвигается еще на один шаг SplitMix64. Из результата вычисляется целое число:

```
seed7 = 1 + avalanche(external_random_state) mod 9999999
```

Диапазон `seed7` равен от 1 до 9 999 999. Число дополняется ведущими нулями до семи цифр и трактуется как логическое значение:

```
0.d1d2d3d4d5d6d7
```

Нулевое слово 0.0000000 намеренно исключено.

## 4. Запись в скрытое слово `xi`

ST IK1306 - транзитный рабочий массив, а не постоянный регистр `xi`. Только в единственный проверенный момент перед A7 слово `xi` выровнено по приведенным ниже позициям. Callback срабатывает после определения адреса, но до декодирования ROM-команды, и записывает цифры в обратном порядке, начиная с младшей:

| Индекс ST | Значение |
|-----------|----------|
| 1         | 0        |
| 4         | d7       |
| 7         | d6       |
| 10        | d5       |
| 13        | d4       |
| 16        | d3       |
| 19        | d2       |
| 22        | d1       |
| 25        | 0        |
| 28        | 9        |
| 31        | 9        |
| 34        | 9        |
| 37        | 0        |
| 40        | 0        |

Три служебные тетрады 999 входят в корректный формат внутреннего слова `xi`. Такая раскладка дает допустимое BCD-число и не создает ЕГГОГ или сверхчисло.

Запись выполняется через `context.st` только в скрытый ST микросхемы IK1306. Callback не меняет регистры X, Y, Z, T, память, стек пользователя или индикатор.

Вне точки A7 эта раскладка неверна. Следующий микрошаг сразу потребляет данные; seed не хранится в ST после старта и не записывается из произвольного места основного цикла.

## 5. Завершение штатной ROM-командой

Сразу после подстановки эмулятор продолжает обычный микрошаг. Командное слово ROM в точке A7 читает новое `x1`, продолжает алгоритм пользовательской команды `0x3B` и штатным путем помещает результат в X и на индикатор.

Поэтому пользователь видит именно результат алгоритма МК-61, запущенного от нового скрытого состояния. Прошивка не записывает готовое псевдослучайное число в X и не подменяет поведение остальных команд.

## Почему подстановка выполняется при каждом К СЧ

Одной подстановки после старта недостаточно. В измеренной последовательности штатного алгоритма для проверенного начального состояния обнаружены 26 значений до входа в цикл и цикл длиной 153 значения, всего 179 различных значений до повтора.

Если дать внешний `x1` только первому вызову, дальнейшая последовательность снова будет развиваться только штатным алгоритмом и может попасть в короткий цикл. Поэтому в режиме МК61s свежее семизначное `x1` подставляется перед каждым выполнением К СЧ, а ROM каждый раз преобразует его обычным способом.

## Почему ROM не патчится

Таблицы ROM остаются без изменений. Реализация регистрирует callback общего ROM-command API в точно определенном внутреннем состоянии IK1306:A7.

Это сохраняет несколько полезных свойств:

- режим МК61 может полностью отключить расширение;
- вся арифметика и видимый эффект команды остаются штатными;
- нет необходимости размещать новый генератор в микрокоде исторической ROM;
- изменение изолировано от остальных команд эмулятора;
- тот же диспетчер можно использовать для независимых расширений других ROM-команд.

Фактически расширяется источник скрытого состояния `x1`, а не заменяется команда К СЧ.

## FOCAL и TinyBASIC

FOCAL и TinyBASIC не вызывают К СЧ эмулируемого калькулятора. Их `RND()` берет старшие 24 бита очередного 32-битного значения своего домена и делит их на `16777216.0`:

```
RND = (next_u32(domain) >> 8) / 16777216.0
```

Результат находится в диапазоне `[0, 1)`. Оба языка всегда используют аппаратно инициализированные независимые потоки, даже когда в настройках выбран К СЧ МК61.

При сборке host-тестов вместо аппаратного пула используются фиксированные тестовые генераторы. Это сделано для воспроизводимости автоматических тестов и не относится к прошивке устройства.

## Сохранение состояния при внутренних вычислениях

При математическом backend CORE FOCAL может временно использовать ядро МК-61 для вычисления функций. Снимок контекста включает не только регистры и память эмулятора, но и `external_random_enabled` с `external_random_state`.

После внутреннего вычисления контекст восстанавливается. Поэтому служебный вызов математической функции не сдвигает пользовательскую последовательность К СЧ МК61s.

## Проверки

Автоматические проверки подтверждают следующие свойства:

- несколько hook могут быть зарегистрированы одновременно;
- callback одной команды выполняются в порядке регистрации;
- замена replacement\_address доходит до следующего callback;
- удаление по handle не затрагивает остальные регистрации;
- ГСЧ включается даже при заполненных восьми пользовательских слотах;
- в режиме MK61 разные внешние seed не меняют первый штатный результат;
- в режиме MK61s разные seed меняют результат К СЧ;
- результат остается в диапазоне  $[0, 1)$ ;
- обычная клавиша не раскрывает скрытый поток;
- временное использование ядра математическим backend не сдвигает поток;
- в выборке из 256 значений расширенный режим не наследует короткий штатный цикл.

При проверке на устройстве штатный режим после перезапуска повторял первое значение 0.4040670. В режиме MK61s после двух перезапусков наблюдались разные первые значения, например 0.3381330 и 0.9296102.

В дополнительной выборке из 10 000 вызовов расширенного режима получено 9957 различных семизначных результатов. Среднее составило приблизительно 0.4949, а распределение по десяти равным интервалам было близко к равномерному. Это полезная инженерная проверка, но не доказательство криптографического качества.

## Режим совместимости MK61

Когда выбран К СЧ MK61, external\_random\_enabled имеет значение false и hook сразу завершает работу. ROM, скрытое слово xi и все последующие переходы обрабатываются так же, как в исходном эмуляторе.

Таким образом, после холодного старта этот режим нужен для программ, тестов и демонстраций, которым важна исторически воспроизводимая последовательность. Режим MK61s предназначен для случаев, где повтор одного и того же старта и короткий цикл нежелательны.

## Связанные файлы исходного кода

Основная реализация находится в code/entropy\_pool.{hpp,cpp} и code/mk61emu\_core.cpp; стартовый сбор и настройки - в code/mk61s-M.ino, code/startup\_splash.cpp, code/keyboard.cpp, code/menu.cpp, code/tools.hpp; языковые RND() - в code/focal.cpp, code/tinybasic.cpp; проверки - в tests/mk\_math\_self\_test.cpp.